1. KAMPUSKIT App Documentation

Project Overview

The KAMPUSKIT App is a module within the larger KAMPUSKIT student productivity platform. Its purpose is to let students create, organize, and manage notes and to-do lists in one place. In context, this Notes App integrates with other KAMPUSKIT tools (like an AI resume builder, chatbot, and PDF summarizer) to provide a unified experience. For example, a student can capture lecture notes and to-do tasks here, then switch seamlessly to other KAMPUSKIT features without re-authenticating. This one-time login or Single Sign-On behavior means users "authenticate once and gain access to multiple applications" (i.e. no repeated logins for each tool).

Features and Functionalities

- Note Management (CRUD): Users can create, read, update, and delete notes. Each note has a title, content, and optional tags.
- To-Do Lists: In a separate section, users can manage to-do lists. A to-do list has its own title and an array of items. Items can be added via a "+ Add Item" button and checked off when completed. To-dos can also be created, updated, and deleted.
- Create Mode Toggle: The Create button toggles between Note and To-Do modes. When creating, the user chooses whether to add a standard note (title + free-text content) or a to-do list (title + checklist items).
- Tagging: Notes (and to-do lists) can be tagged for organization. Tags are entered as comma- separated values in a tag field. These tags allow grouping and filtering of related notes.
-

Single Sign-On (SSO): KAMPUSKIT enforces a single login across all modules. Once the user signs in, they can access the Notes App and all other tools without additional logins . This SSO approach improves security and usability.

Technical Stack

The Notes App uses the MERN stack (MongoDB, Express, React, Node) – a full-stack JavaScript framework . Each layer is:

-
Frontend: React.js – a JavaScript library for building user interfaces . React renders the Notes and To-Do UI components in the browser, handling user input and navigation.
-
Backend: Node.js with Express.js – Node provides an asynchronous, event-driven runtime for server code , and Express is a minimal, flexible web framework on Node for building RESTful APIs . The backend exposes HTTP endpoints for notes and to-dos (CRUD operations) and handles requests from the React frontend.
- Database: MongoDB – a document-oriented NoSQL database. MongoDB stores notes and to-do lists

as flexible JSON-like documents . Its schema-less nature (collections of documents) fits well with storing user-generated content (notes with tags, lists of to-do items, etc.).

This three-tier architecture (React front-end, Express/Node app server, MongoDB database) follows a classic “MERN” structure . All code is written in JavaScript/TypeScript, simplifying development across client and server.

User Authentication

KAMPUSKIT implements Single Sign-On (SSO): the user logs in once via the authentication service and gains access to all KAMPUSKIT modules (Notes, Resume Builder, Chatbot, etc.) without logging in again

. In practice, the backend might use OAuth2/OpenID or JSON Web Tokens (JWT) to manage sessions. For example, Passport.js could be used to integrate an identity provider (Google, etc.) so that a token or session cookie works across services . In the Notes App, protected API endpoints check for an authenticated session or valid token. A middleware (e.g.,

) verifies the user’s identity before allowing note/todo operations (illustrated in [16] with Passport.js). This ensures efficient management and enhanced security (users aren’t prompted repeatedly for credentials).

Problem: Backend connection failed

l Cause:

FastAPI server not running, incorrect API endpoint, or CORS configuration issue.

l Solution:

1.Start backend using: uvicorn main:app --reload
2. Verify frontend API URL is correct.
3. Ensure CORS middleware is enabled.
4. Check terminal logs for errors.

l Technical Analysis:

The frontend sends POST requests to /chat endpoint. If server is not active or blocked by CORS, the request fails.

Problem: Chatbot says 'I don't know'

l Cause:

The requested answer is not present in the uploaded documentation used for vector indexing.

l Solution:

1. Ensure consolidated documentation PDF is loaded.
2. Rebuild vector store.
3. Restart backend server.

l Technical Analysis:

The RAG system retrieves top-k similar chunks. If the information is not embedded in FAISS, the model cannot answer.

2. AI Learning Assistant

What is it?

A smart study platform that transforms PDF documents into interactive learning experiences using AI.

Who is it for?

College students who want to study more efficiently and understand their textbooks, lecture notes, and research papers better.

Main Purpose:

Instead of just reading PDFs, students can chat with AI about the content, get summaries, create flashcards, and take quizzes — all automatically generated from their documents.

Core Features
1. User Account System

What users can do:

Create a personal account with email and password

Log in securely to access their study materials

Keep all their documents, flashcards, and quizzes private

How it works:

Click "Sign Up" and enter name, email, password (minimum 6 characters)

Account created instantly

Automatically logged in

Stay logged in for 7 days (then need to log in again for security)

2. PDF Document Upload

What users can do:

- Upload PDF, textbooks, lecture notes, research papers
- Store unlimited documents (up to 10MB each)
- View uploaded documents directly in browser
- Organize and manage their document library
- Delete documents when they no longer need

How it works:

- Click "Upload Document" on dashboard
- Select PDF file from computer (max 10MB)
- System automatically extracts text from PDF (takes a few seconds)
- Document appears in your library, ready to use

Important Notes:

- Only PDF files are supported (no Word docs, images, etc.)
- Best with text-based PDFs (not scanned/image PDFs)
- 10MB size limit per file
- Text extraction is automatic — needed for AI features

3. AI-Powered Document Chat

What users can do:

- Ask questions about their uploaded document
- Get instant, intelligent answers from AI
- Have natural conversations about the content
- Get explanations, summaries, or clarifications
- Conversation history saved automatically

How it works:

- Open any uploaded document
- Click "Chat with AI" button
- Type your question in the chat box
- AI reads your document and provides context-aware answer

Continue asking follow-up questions

All conversations saved per document

4. Instant Document Summary

What users can do:

Generate a concise summary of entire document with one click

Get key points and main ideas highlighted

Save summary for future reference

Review before reading the full document

How it works:

Open any document

Click "Generate Summary"

AI reads entire document (takes 10–20 seconds for long docs)

Summary appears on screen

Summary automatically saved with document

5. Concept Explainer

What users can do:

Request detailed explanation of specific topics

Get simplified versions of complex concepts

See examples and analogies

Understand context-specific meanings

How it works:

Open document with concept you don't understand

Click "Explain Concept" or use chat

Type concept/topic

AI provides: Definition → Detailed Explanation → Example → Context

6. Auto-Generated Flashcards

What users can do:

- Automatically create flashcard sets from any document
- Choose number of flashcards (5 to 50)
- Flip cards to reveal answers
- Mark important cards as favorites
- Delete unwanted cards
- Filter to show only favorite cards

How it works:

- Open document
- Click "Generate Flashcards"
- Choose number (5–50)
- AI analyzes document and creates Q/A pairs
- Takes 5–10 seconds
- Cards appear with flip animation
- Click to flip and see answer
- Navigate with next/previous
- Click heart icon to mark favorites

7. AI Quiz Generator

What users can do:

- Generate custom multiple-choice quizzes
- Choose number of questions (5–20)
- Take quiz and get instant feedback
- See detailed explanations
- Track quiz history and scores

How it works:

- Open document
- Click "Generate Quiz"
- Choose 5–20 questions

- AI creates MCQs (10–15 seconds)
- Each question has 4 options
- Submit answers
- See score and explanations

8. Progress Dashboard

Users see:

- Total documents uploaded
- Total flashcards created
- Total quizzes completed
- Average quiz score
- Recent activity
- Quick access to documents

9. Favorites System

- Star important flashcards
- Filter to only favorites
- Quick review of difficult concepts
- Toggle favorites easily

10. Study History & Analytics

- Quiz scores
- Documents studied
- Study time
- Performance trends
- Best/worst topics

Complete User Workflows

Workflow A: First Study Session

Create account → Upload document → Generate summary → Ask questions → Create flashcards

Workflow B: Exam Preparation

Review flashcards → Read summary → Take quiz → Review mistakes → Final AI Q&A

Workflow C: Difficult Topic

Upload notes → Get explanation → Ask follow-ups → Create focused flashcards → Test understanding

Workflow D: Quick Document Scan

Upload → Generate summary → Ask specific questions → Decide relevance

Problem: PDF upload failed

l Cause:

File exceeds 10MB, not a PDF, or scanned image-based PDF.

l Solution:

1. Upload text-based PDF under 10MB.
2. Convert scanned PDF using OCR tool.
3. Retry upload.

l Technical Analysis:

Text extraction requires readable text layer. Image-based PDFs lack extractable text.

Problem: AI response is slow

l Cause:

Large document size or API latency.

l Solution:

1. Wait 10–20 seconds.
2. Try smaller document.
3. Check internet speed.

l Technical Analysis:

Large context increases embedding and retrieval processing time.

3. SGPA & CGPA Calculation

What is this project?

A simple web-based calculator that allows students to compute:

SGPA (Semester Grade Point Average) for the current semester

CGPA (Cumulative Grade Point Average) across all semesters

The application is built using HTML, CSS, and JavaScript and runs entirely in the browser.

Who is it for?

College and university students

Anyone needing quick SGPA/CGPA calculation

Academic institutions for simple GPA tool

3. Application Structure

The project consists of three files:

3.1 index.html

index

Defines:

Page title

SGPA section

CGPA section

Buttons for adding inputs

Result display areas

Links to CSS and JS

Two main sections:

SGPA Section

Grade Point input

Credits input

Add Subject button

Calculate SGPA button

Result display paragraph

CGPA Section

Semester SGPA input

Add Semester button

Calculate CGPA button

Result display paragraph

3.2 script.js

script

Contains all logic functions:

SGPA Functions

addSubject()

Dynamically creates:

Grade input field

Credit input field

Appends them to the page

calculateSGPA()

Formula Used:

SGPA=∑(GradePoint×Credit)∑CreditsSGPA = \frac{\sum (GradePoint × Credit)}{\sum Credits}SGPA=∑Credits∑(GradePoint×Credit)

Logic:

Select all subject inputs

Loop in pairs (grade, credit)

Multiply and accumulate

Divide totalPoints by totalCredits

Display result (rounded to 2 decimals)

Validation:

Ignores invalid or zero inputs

Shows error if no valid data entered

CGPA Functions

addSemester()

Adds new SGPA input field dynamically

calculateCGPA()

Formula Used:

CGPA=∑SemesterSGPANumber of SemestersCGPA = \frac{\sum SemesterSGPA}{Number\ of\ Semesters}CGPA=Number of Semesters∑SemesterSGPA

Logic:

- Collect all semester SGPA inputs
- Sum valid values
- Divide by count
- Display result (rounded to 2 decimals)

3.3 style.css

style

Defines:

- Centered layout
- Max width 700px
- Styled buttons
- Input spacing
- Box containers
- Result text styling (bold, green)

UI Features:

- Clean minimal layout
- Box separation for SGPA and CGPA
- Rounded borders
- Proper spacing

4. Core Features

4.1 SGPA Calculator

What Users Can Do:

- Enter multiple subjects
- Dynamically add subjects

Calculate weighted SGPA

See formatted result instantly

4.2 CGPA Calculator

What Users Can Do:

Enter multiple semester SGPAs

Add unlimited semesters

Calculate overall CGPA

View rounded result

5. User Workflow

Workflow A: Calculate SGPA

Enter grade point

Enter credits

Click “Add Subject” for more

Click “Calculate SGPA”

View result

Workflow B: Calculate CGPA

Enter semester SGPA

Click “Add Semester” for more

Click “Calculate CGPA”

View result

6. Validation & Error Handling

SGPA Errors:

If totalCredits == 0
→ Displays error message

CGPA Errors:

If no semester entered
→ Displays error message

Current validation checks:

Values must be > 0

Empty fields ignored

7. Mathematical Correctness

SGPA uses Weighted Average

Correct formula implemented:

∑(GP×Credit)∑Credit\frac{\sum (GP × Credit)}{\sum Credit}∑Credit∑(GP×Credit)

Verified:

Loop increments by 2 (correct pairing)

Credit weight applied properly

Decimal precision fixed to 2

CGPA uses Simple Average

∑SGPAn\frac{\sum SGPA}{n}n∑SGPA

Verified:

Only valid values counted

Correct division by count

Problem: SGPA shows 0

l Cause:

Total credits entered equals zero.

l Solution:

Ensure at least one subject has valid grade and credit (>0).

l Technical Analysis:

Weighted average formula divides by total credits. Zero denominator invalidates result.

Problem: CGPA error shown

l Cause:

No semester SGPA values entered.

l Solution:

Enter at least one semester SGPA before calculation.

l Technical Analysis:

CGPA calculation requires at least one valid numeric entry.

4. Resume Builder

1. Project Overview

What it is: The AI Resume Builder is a full-stack web application built as part of a larger student productivity platform. It is designed for college students, freshers, and entry-level job seekers to create professional, ATS-ready resumes quickly — with the help of artificial intelligence.

Most students struggle with resumes because they lack writing experience or do not know how to structure their information. This tool solves that by providing guided form sections, real-time previews, multiple templates, and AI assistance for content writing.

Who is it for:

- College students applying for internships
- Fresh graduates applying for their first job
- Entry-level job seekers who need a professional resume fast
- Anyone who wants to quickly update or digitise an existing resume

Core Value Propositions:

- AI-assisted content — descriptions, summaries, and job roles enhanced by AI
- PDF upload and auto-fill — upload an existing resume and have all fields populated automatically
- Multiple templates — choose a resume style that fits your industry
- ATS-ready output — structured to pass Applicant Tracking Systems
- One-click download — save your resume as a PDF directly from the browser
- Public sharing — generate a shareable link to your resume for recruiters

2. User Authentication

What it is: The application uses JWT-based authentication. Users register with their name, email, and password. Passwords are hashed using bcrypt before storing. On login, a signed JWT token is returned and stored in the browser's localStorage.

How it works:

1. User registers or logs in via the Login/Register page.
2. The backend validates credentials and returns a signed JWT token.
3. The token is stored in localStorage and attached to every API request via the Authorization header.
4. On app startup, the token is retrieved and validated via GET /api/users/data.
5. If valid, user state is loaded into Redux and the Dashboard is shown.
6. If invalid or expired, the user is redirected to the Login page.

Auth Middleware:

All protected API routes pass through the protect middleware. It extracts the token from req.headers.authorization, verifies it with the JWT_SECRET, and attaches the decoded userId to the request object. If missing or invalid, a 401 Unauthorized response is returned immediately

3. Resume Dashboard

What it is: The Dashboard is the central hub after login. It displays all resumes as visual cards and provides buttons to create a new resume or upload an existing one. Cards show the resume title, last updated date, and quick action icons.

What users can do:

View all resumes as colour-coded cards at a glance

- Create a new blank resume by entering a title
- Upload an existing PDF resume for AI-powered auto-fill
- Click any resume card to open it in the Resume Builder
- Hover over a card to reveal the pencil (rename) and trash (delete) icons
- Rename a resume using the pencil icon
- Delete a resume with a confirmation prompt

Resume Card Appearance:

Each card is 192px tall with a colour-coded icon and resume title. The border and icon colour cycles through 5 preset colours — purple, amber, red, sky blue, and green — based on the card's position. The last updated date is shown at the bottom of each card.

Key Benefits:

- All resumes visible at a glance — no nested menus
- Multiple resumes can be maintained simultaneously (e.g., one for tech roles, one for internships)
- Direct navigation to any resume with a single click

4. Resume Builder — Form Editor

What it is: The Resume Builder is a split-screen page. The left panel is the form editor with six structured sections. The right panel shows a live resume preview that updates in real time as the user types.

Layout:

- Desktop: left panel (5/12 width) = form editor | right panel (7/12 width) = live preview
- Mobile: form appears above the preview in a stacked layout
- A progress bar at the top of the form fills as the user moves through sections

Section Navigation:

The form has 6 sections navigated using Previous and Next buttons: Personal Info → Professional Summary → Experience → Education → Projects → Skills

Top Toolbar Controls:

- Template Selector — choose from 4 resume templates
- Color Picker — choose an accent color applied across the entire resume
- Previous / Next buttons — navigate between the 6 form sections

Save Changes Button:

Located at the bottom of the form panel. Sends a PUT request to /api/resumes/update which saves all current data — sections, template, accent color, visibility — to the database. If a new profile photo is selected, it is also uploaded to ImageKit during this save.

Right Panel Action Buttons:

- Private / Public toggle — switches the resume between private and publicly accessible

- Share — appears only when Public; triggers the Web Share API to share the resume URL
- Download — calls window.print() with print-optimised CSS to save as PDF

5. Resume Sections — Detailed Breakdown

5.1 Personal Information

- Full Name — displayed as the headline of the resume
- Profession / Job Title — shown below the name in the header
- Email Address
- Phone Number
- Location (City, Country)
- Website URL — portfolio or personal site
- LinkedIn Profile URL — shown without https://www. prefix in templates
- Profile Photo — optional upload, processed and hosted via ImageKit

5.2 Professional Summary

A free-text area for a short paragraph (3–5 sentences) summarising the candidate's background and goals. An AI button lets users auto-generate or improve this text. AI endpoint: POST /api/ai/enhance- pro-sum.

5.3 Work Experience

Multiple entries supported. Each entry contains:

- Company Name
- Job Title / Position
- Start Date and End Date (month pickers)
- Currently Working Here checkbox — disables the end date field when checked
- Job Description — multi-line textarea with an 'Enhance with AI' button

The AI Enhancement button sends the company, position, and description to POST /api/ai/enhance- job-desc. The AI rewrites it with stronger action verbs and clearer impact. The button is disabled until company and position are filled.

5.4 Education

Multiple entries supported. Each entry contains: Institution Name, Degree (e.g. B.Tech), Field of Study, Graduation Date (month picker), and optional GPA.

5.5 Projects

Multiple entries supported. Each project contains: Project Name, Project Type (e.g. Web App, API, Research), and a Description of what was built and the technologies used.

5.6 Skills

Users add individual skills as text tags. Skills are stored as a simple array of strings. In the resume preview, each skill appears as a coloured pill badge using the selected accent color.

6. AI-Powered Features

The application integrates AI across three distinct features. All AI endpoints are protected and require a valid JWT token.

6.1 AI Resume Upload & Parsing — POST /api/ai/upload-resume

When a user uploads a PDF resume from the Dashboard:

7. The PDF is parsed client-side using react-pdftotext, extracting all raw text.
8. The extracted text and resume title are sent to the backend endpoint.
9. The backend sends the text to the AI model with a prompt to extract and organise the content into the application's data schema (personal info, experience, education, projects, skills, summary).
10. The structured AI response (JSON) is used to create a new resume document in MongoDB.
11. The user is redirected to the builder with all sections pre-populated and ready to edit.

6.2 Professional Summary Generation — POST /api/ai/enhance-pro-sum

In the Professional Summary section, the user can type a rough draft or leave it blank and click the AI button. The content is sent to the AI which returns a polished, recruiter-friendly summary paragraph. The user can edit the result as needed.

6.3 Job Description Enhancement — POST /api/ai/enhance-job-desc

Available for every experience entry. The prompt is: "Enhance this job description [description] for the position of [position] at [company]." The AI returns an improved description with stronger action verbs and impact statements, replacing the original text.

Key Benefits:

- Saves time for users who do not know what to write
- Produces professional, recruiter-friendly language automatically
- Converts vague descriptions into clear achievement statements
- Improves ATS scores by using industry-appropriate keywords

7. Resume Templates

Four professionally designed templates are available. The template can be changed at any time from the Template Selector in the builder. The preview updates instantly and the choice is saved with the resume.

Available Templates:

- 1. Modern (default) — Contemporary design with a full-width coloured header showing name and contact info. Experience and projects use a left-border timeline. Education and Skills are in a two-column grid. Best for: tech, design, and creative roles.
- 2. Classic — Traditional formal layout with a centred header, clear dividers, and a linear structure. Best for: finance, law, consulting, and corporate roles.
- 3. Minimal — Whitespace-heavy, typography-focused design. Best for: academic, research, and roles where simplicity signals professionalism.
- 4. Minimal with Image — Same as Minimal but includes a circular profile photo in the header. Best for: creative fields or roles where a personal touch is valued.

How Templates Work Technically:

Each template is a separate React component (ModernTemplate, ClassicTemplate, MinimalTemplate, MinimalImageTemplate) that receives the full resume data and accentColor as props. The ResumePreview component renders the correct template based on the template field in state. Switching is instant with no data loss.

8. Accent Color Customization

What it is: Users choose a custom accent color applied across the entire resume — header backgrounds, company names, skill pill badges, section borders, and highlights — all update to the chosen color instantly.

How it works: The ColorPicker component lets users pick any color. The hex value is stored in resumeData.accent_color and passed as a prop to the active template. The preview re-renders immediately. The color is saved to the database on Save Changes.

9. Profile Photo Upload & AI Background Removal

What it is: Users upload a profile photo from the Personal Info section. The image is automatically processed by ImageKit — face-cropped to a square — and the CDN URL is saved to the resume and shown in compatible templates.

Image Processing Pipeline:

12. User selects a photo in the Personal Info form.
13. The image file is held in frontend state (not yet uploaded).
14. On Save Changes, the image is sent as multipart form data to the backend via multer.
15. The backend streams the image to ImageKit with transformation: w-300, h-300, fo-face (face- centred crop), z-0.75.
16. If Background Removal is enabled, the e-bgremove transformation is applied using ImageKit's AI engine.
17. The returned CDN URL is stored in personal_info.image and saved to the database

10. Resume Visibility & Public Sharing

What it is: All resumes are Private by default. Users can toggle a resume to Public, making it accessible via a unique shareable URL without requiring the viewer to log in.

How it works:

- The toggle button is in the right panel of the Resume Builder
- Clicking it sends PUT /api/resumes/update with the updated public value
- When Public, a Share button appears — generates [frontend-url]/view/[resumeId] and triggers the Web Share API
- Anyone with the link can view the resume at /view/:resumeId with no login required

- The public view fetches via GET /api/resumes/public/:resumeId, which only returns resumes where public: true

11. Download & Print

What it is: Users download their resume as a PDF via the Download button. This uses the browser's print functionality with carefully crafted CSS print rules to produce a clean PDF output.

How it works:

- The Download button calls window.print() which opens the browser print dialog
- CSS @media print rules hide everything except the #resume-preview element
- The resume is repositioned to fill the page with no extra margins or borders
- Page size is set to US Letter (8.5 x 11 in) via the CSS @page rule
- The user selects 'Save as PDF' in the print dialog to download

12. Resume Data Structure

Each resume is stored as a single document in MongoDB using the following Mongoose schema:

Top-Level Fields:

Field	Type	Description
userId	ObjectId (ref: User)	Owner of the resume
title	String	Resume display name — default: 'Untitled Resume'
public	Boolean	Publicly accessible or not — default: false
template	String	'modern', 'classic', 'minimal', or 'minimal-image'
accent_color	String (hex)	Accent color — default: '#3B82F6'

professional_summary	String	The summary paragraph
skills	String[ ]	Array of skill tag strings
personal_info	Object	See personal_info sub-fields below
experience	Object[ ]	Array of experience entries — see below
education	Object[ ]	Array of education entries — see below
projects	Object[ ]	Array of project entries — see below
createdAt / updatedAt	Date	Auto-generated Mongoose timestamps

personal_info Sub-Fields:

- image — ImageKit CDN URL of the processed profile photo
- full_name — displayed as the resume headline
- profession — job title shown below the name
- email, phone, location, website

experience Entry Fields:

- company, position — employer name and job title
- start_date, end_date — format: YYYY-MM
- description — responsibilities and achievements
- is_current — Boolean, true if currently employed (end_date is ignored)

education Entry Fields:

- institution, degree, field — school name, degree type, field of study
- graduation_date — format: YYYY-MM
- gpa — optional grade point average

projects Entry Fields:

- name — project title
- type — category (e.g. Web App, API, Research)
- description — what was built and the technologies used

13. Key User Flows

Flow 1: Create a Resume from Scratch

18. User visits the landing page and clicks 'Get Started'.
19. User registers an account and lands on the empty Dashboard.
20. Clicks 'Create Resume', enters a title, and confirms.
21. Redirected to the Resume Builder with a blank form.
22. Fills in Personal Info; optionally uploads a profile photo.
23. Navigates to Professional Summary and uses the AI button to generate text.
24. Adds Experience entries; clicks 'Enhance with AI' for each job description.
25. Fills in Education, Projects, and Skills sections.
26. Selects a template and accent color; watches the preview update live.
27. Clicks 'Save Changes' to persist the resume to the database.
28. Clicks 'Download' to save the resume as a PDF.

Flow 2: Upload an Existing Resume

29. User clicks 'Upload Existing' on the Dashboard.
30. Modal opens — user enters a title and selects a PDF file.
31. Loading spinner appears while the PDF is parsed and sent to the AI.
32. User is redirected to the builder with all sections pre-filled.
33. User reviews and edits the pre-filled data, saves, and downloads.

Flow 3: Share a Resume Publicly

34. User opens a resume in the builder.
35. Clicks the 'Private' toggle — it switches to 'Public'; a Share button appears.
36. Clicks Share — the Web Share API generates and shares the resume URL.
37. User sends the link via WhatsApp, email, LinkedIn, etc.
38. Recipient opens /view/:resumeId in their browser — no login required.

Problem: Cannot login

l Cause:

Incorrect password or expired token.

l Solution:

1. Re-enter correct credentials.
2. Clear localStorage.
3. Login again.

l Technical Analysis:

JWT authentication validates token on protected routes. Invalid tokens cause 401 error.

Problem: AI enhance button disabled

l Cause:

Company name or Job Title not filled.

l Solution:

Fill required fields before clicking Enhance with AI.

l Technical Analysis:

Frontend validation prevents API call if mandatory fields are missing.

Problem: Resume not downloading

l Cause:

Browser blocked print dialog.

l Solution:

Allow pop-ups and select 'Save as PDF' in print dialog.

l Technical Analysis:

Download uses window.print() with CSS print styling.

Problem: Public link not working

l Cause:

Resume visibility set to Private.

l Solution:

Toggle resume visibility to Public before sharing.

l Technical Analysis:

Public route only returns resumes where public flag = true.

5 . Video Summarization Documentation

1. VIDEO FILE SUMMARIZATION (Upload-Based)

Feature Name

Video File Summarization

Purpose

Allows users to upload a local video file and automatically generate a textual summary of its content.

Navigation Path

Application → Video Section → Video Summarization Block

UI Components Involved

- Page Title: “Video Section”
- Section Heading: “Video Summarization”
- Instruction Text: “Upload video file”
- File Upload Area:
    - Drag-and-drop box
    - “Browse Files” button
- Output Area (Summary display section)
- Optional Toggle: “Enable YouTube Video Question Answering” (Not required for this feature)

Step-by-Step Workflow

Step 1 – Access the Feature

1. Open the application.
2. Navigate to the “Video Section”.

Step 2 – Locate Upload Interface

1. Find the “Video Summarization” block.
2. Identify the file upload component.

Step 3 – Upload Video File

Users can upload video using either method:

Method A – Drag and Drop

- Drag supported video file (.mp4, .mov, .avi, etc.)

- Drop inside upload box.

Method B – Browse Files

- Click “Browse Files”
- Select video from local system
- Click “Open”

Step 4 – Upload Completion

- File uploads to system.
- UI may show file name or loading indicator.

Step 5 – Automatic Processing

After upload:

- System extracts audio from video.
- Speech-to-text transcript is generated.
- Summarization model processes transcript.
- Summary is generated automatically.

Step 6 – Output Display

- Generated summary appears below upload section.
- User can read the summarized content.

Important Rule

The “Enable YouTube Video Question Answering” toggle must remain OFF when using file-based summarization.

Functional Summary

User uploads video → System extracts transcript → Model generates summary → Summary displayed.

2.YOUTUBE VIDEO QUESTION ANSWERING (YouTube-Based)

Feature Name

YouTube Video Question Answering

Purpose

Allows users to input a YouTube video link and ask questions about its content using transcript-based AI processing.

Navigation Path

Application → Video Section → Enable YouTube Video Question Answering

UI Components Involved

- Page Title: “Video Section”
- Toggle: “Enable YouTube Video Question Answering”
- URL Input Field (appears after toggle ON)
- Load / Process Button
- Question Input Field
- Ask / Submit Button
- Answer Output Section

Step-by-Step Workflow

Step 1 – Access the Feature

1. Open the application.
2. Navigate to “Video Section”.

Step 2 – Enable YouTube Mode

1. Locate “Enable YouTube Video Question Answering” toggle.
2. Turn the toggle ON.

Step 3 – Provide YouTube Link

1. Copy the YouTube video URL.
2. Paste it into the URL input field.

Step 4 – Load Video Data

1. Click Load / Process button.
2. System retrieves transcript from YouTube video.

Step 5 – Ask Question

1. Type a question related to the video content.
2. Example: “What is the main topic discussed?”

Step 6 – Submit Question

1. Click Ask / Submit button.

Step 7 – View Answer

1. System processes transcript using QA model.
2. Generated answer appears below question box.

YouTube Video Summarization Using QA

This feature can also be used to summarize a YouTube video.

Method:

1. Enable YouTube Video Question Answering.
2. Paste YouTube link.

3. Load the video.
4. In question box, type:
   "Summarize the video."
5. Click Ask.
6. System generates a full summary.

Important Distinction

- File Upload Summarization → Works only with locally uploaded videos.
- YouTube QA Summarization → Works only with YouTube links.
- The two features operate independently.

✅ Final Clean Summary for Embedding

Video File Summarization:
User uploads local video → System extracts transcript → AI generates summary → Output displayed.

YouTube Video Question Answering:
User enables toggle → Pastes YouTube link → System fetches transcript → User asks question → AI generates answer.

YouTube Video Summarization:
User asks "Summarize the video" inside YouTube QA → AI generates full summary.

6. PDF Summarization Documentation

BART Local Model | No External API

TABLE OF CONTENTS

---

1. Feature Overview
2. How It Works
3. Features In Detail
4. Technical Architecture
5. Project Folder Structure
6. Environment Setup
7. Known Limitations
8. Q&A for Chatbot
9. Platform Integration Notes

SECTION 1 — FEATURE OVERVIEW

WHAT IS THE PDF SUMMARIZER?

The PDF Summarizer is a web-based tool built as part of the College Student Productivity Suite. It allows students to upload
any PDF document and receive an AI-generated structured summary broken into sections with headings — all processed locally using
the BART-large-CNN model, with no external API required.

WHAT PROBLEM DOES IT SOLVE?

College students regularly deal with overwhelming volumes of reading material — research papers, lecture notes, assignment briefs, and journal articles. The PDF Summarizer removes the time burden of reading everything in full before understanding what a document contains. It is especially useful during exam season, literature reviews, or when scanning multiple sources.

TARGET USERS

- Students who need to quickly understand a research paper
- Students preparing for exams by reviewing lecture-note PDFs
- Students conducting literature reviews for dissertations
- Any platform user uploading study material in PDF format

AI ENGINE

This feature uses Facebook AI Research's BART-large-CNN model (Bidirectional and Auto-Regressive Transformer), a powerful abstractive summarization model. It runs entirely on the local server. Documents are never sent to any external service

SECTION 2 — HOW IT WORKS

STEP-BY-STEP USER FLOW

1. Student opens the PDF Summarizer on the platform
2. Drags and drops a PDF onto the upload zone or clicks browse
3. Selects a summary length: Short, Medium, or Long
4. Clicks the "Summarize PDF" button
5. React frontend sends the file to the Express backend
6. Express forwards the PDF to the Flask ML service
7. Flask extracts text using PyPDF2 and detects headings
8. BART tokenizes text into 900-token chunks and summarizes each
9. Chunk summaries are merged into a final structured output
10. Output is formatted into a PDF using ReportLab and returned
11. Summary record is saved to MongoDB with metadata
12. Student receives a downloadable summary PDF

WHAT DOES THE OUTPUT LOOK LIKE?

The summary is divided into named sections with headings that reflect the original document structure. A research paper might produce sections: Introduction, Methodology, Results, and Conclusion — each with a concise summarized paragraph.

PROCESSING TIME (CPU-Based)

1–5 pages → 30–60 seconds
5–15 pages → 1–2 minutes
15–30 pages → 2–4 minutes
30+ pages → 4+ minutes (risk of timeout)

Note: BART runs on CPU. A GPU server would reduce times by 10–20x but is not required. All documents remain private.

SECTION 3 — FEATURES IN DETAIL

DRAG AND DROP UPLOAD

Students drag a PDF directly onto the upload zone. The zone highlights with a glowing border on hover. Only PDF files are accepted. Maximum file size is 50 MB. Filename and size are shown after selection with an option to remove and re-select.

ADJUSTABLE SUMMARY LENGTH

Short → Essential key points only (60–150 words per chunk)
Medium → Balanced overview with headings (130–280 words)
[DEFAULT] Long → Detailed comprehensive summary (220–400 words)

HEADING-AWARE STRUCTURED OUTPUT

The system detects headings using heuristics — short lines that are title-cased or ALL CAPS are identified as headings. The summary is then organized section-by-section under

these headings, making it easy to navigate the original document.

DOWNLOAD SUMMARY AS PDF

The summarized output is generated as a formatted PDF using ReportLab with styled headings and body paragraphs. Output filename: summary-[originalfilename].pdf

COPY TO CLIPBOARD

One button click copies all sections (headings + text) to the

clipboard for pasting into notes or documents.

SUMMARY HISTORY PANEL

Every summary is auto-saved to MongoDB. The History sidebar shows:

- Original PDF filename
- Time since generation (e.g. "2h ago", "3d ago")
- Summary length setting used
- Expandable preview of first two sections
- Delete button to remove a record permanently

TOAST NOTIFICATIONS

Non-blocking toast messages appear for: successful summarization, copy confirmation, download confirmation, and error states. Each auto-dismisses after 3.2 seconds.

ERROR HANDLING

Clear messages shown for: scanned/image-based PDF, file too large, server timeout, and general processing failures. UI remains usable after an error without a page refresh.

SECTION 4 — TECHNICAL ARCHITECTURE

SYSTEM DIAGRAM

```
Student Browser  (React 18 + Vite — Port 5173)
      |
      v
Express Backend  (Node.js — Port 5000)
      |              |
      v              v
Flask ML Service   MongoDB
Database (Python — Port 5001)
(History Storage)
BART-large-CNN
(~1.6 GB model, CPU inference)
```

FRONTEND COMPONENTS

```
App.jsx        →  Root — state, API calls, layout
UploadZone.jsx  →  Drag-and-drop file input
LengthControl.jsx →  Short/Medium/Long toggle
SummaryResult.jsx →  Renders sections, copy &
download HistoryPanel.jsx  →  Sidebar with past
summaries useToast.js                    →  Toast
notification hook
```

BACKEND FILES (Node.js / Express)

server.js → Entry point, DB connection, routes routes/summarize.js → POST /api/summarize routes/history.js → GET & DELETE /api/history models/PdfSummary.js → Mongoose schema middleware/upload.js → Multer, 50MB limit, PDF filter

ML SERVICE FILES (Python / Flask)

app.py → Flask entry, loads BART at startup
routes/summarize.py → POST /summarize-pdf endpoint

utils/extractor.py → Text extraction + heading detection
utils/summarizer.py → BART chunking + beam search
inference utils/pdf_writer.py → ReportLab PDF output
generation

MONGODB SCHEMA

PdfSummary
{ originalFileName :
String
originalFileSize : Number (bytes)
summaryFileName : String
summaryLength : "short" | "medium" |
"long" sections : [{ heading: String, text:
String }] pageCount : Number
createdAt : Date (auto)
updatedAt : Date
(auto)
}

API ENDPOINTS

POST /api/summarize Upload PDF, receive
summary PDF GET /api/history Fetch last 50
summaries
DELETE /api/history/:id Delete a summary record

SECTION 5 — PROJECT FOLDER STRUCTURE

```
pdf-summarizer/
├── frontend/
│   └── src/
│       ├── components/
│       │   ├── UploadZone.jsx
│       │   ├── LengthControl.jsx
│       │   ├── SummaryResult.jsx
│       │   └── HistoryPanel.jsx
│       ├── hooks/
│       │   └── useToast.js
│       ├── App.jsx
│       ├── App.css
│       ├── index.css
│       └── main.jsx
│
├── backend/
│   ├── routes/
│   │   ├── summarize.js
│   │   └── history.js
│   ├── models/
```

```
|   |   └── PdfSummary.js
|   ├── middleware/
|   |   └── upload.js
|   ├── server.js
|   ├── package.json
|   └── .env.example
|
└── ml/
    ├── routes/
    |   └── summarize.py
```

```
  ├── utils/
  |   ├── extractor.py
  |   ├── summarizer.py
  |   └── pdf_writer.py
  ├── app.py
  └── requirements.txt
```

SECTION 6 — ENVIRONMENT SETUP

PREREQUISITES

- Node.js v18 or higher
- Python 3.10 or higher
- MongoDB (local or Atlas)
- ~2 GB free disk space for the BART model

STEP 1 — ML Service (Flask, Port 5001)

```
cd ml
pip install -r requirements.txt
python app.py
```

NOTE: On first run, BART-large-CNN (~1.6 GB) is automatically downloaded from HuggingFace. This is a one-time download. Subsequent starts load from disk and are much faster.

STEP 2 — Backend (Express, Port 5000)

```
cd backend
npm install
cp .env.example .env
npm run dev
```

STEP 3 — Frontend (React, Port 5173)

```
cd frontend
npm install
npm run dev
```

ENVIRONMENT VARIABLES (backend/.env)

```
MONGO_URI=mongodb://localhost:27017/pdfSummarizer
PORT=5000
ML_URL=http://127.0.0.1:5001
```

IMPORTANT: MongoDB database name is case-sensitive. Use "pdfSummarizer" (capital S) exactly. Using "pdfsummarizer" will cause a "db already exists with different case" error.

SECTION 7 — KNOWN LIMITATIONS

Scanned PDFs

Cannot read image-based PDFs. Use OCR software to convert scanned documents to text-based PDFs before uploading.

Processing Speed
BART runs on CPU only. A 10-page PDF takes ~1–2 minutes. A GPU server would reduce this to under 10 seconds.

Long Document Timeout
Documents over ~30 pages may hit the 5-minute timeout. Use Short length setting or split the file before uploading.

50 MB File Limit
Files larger than 50 MB are rejected immediately.

Non-English Documents
BART was trained on English text. Quality drops significantly for non-English PDFs.

Mathematical Content
Equations and symbols may not extract or summarize correctly.

First Startup Delay
Flask downloads ~1.6 GB on first run. It will not respond until the download completes. This happens only once.

Heading Detection Accuracy
Headings are detected heuristically. Documents with unusual formatting may not have sections correctly identified

SECTION 8 — Q&A FOR CHATBOT

This section contains all anticipated student questions and their accurate answers. The platform chatbot should use these when students ask about the PDF Summarizer.

Q: What is the PDF Summarizer?
A: It is a tool on this platform that lets you upload any PDF and get an AI-generated summary of its contents. It saves time by automatically extracting and summarizing key points from long documents like research papers, textbooks, and lecture notes.

Q: How do I use the PDF Summarizer?
A: Open the PDF Summarizer page, drag and drop your PDF onto the upload area or click to browse, choose a summary length (Short, Medium, or Long), then click "Summarize PDF". The system will process your document and return a structured summary with section headings.

Q: What file types can I upload?

A: Only PDF files are supported. The file must be under 50 MB.

Word and PowerPoint files are not directly supported — export them to PDF first, then upload.

Q: Why is my PDF not working?
A: The most common reason is that your PDF is scanned or image-based. Try selecting and copying text from your PDF in a viewer. If you cannot select any text, it is image-based and cannot be processed. Use OCR software to convert it first

Q: How long does summarization take?
A: For documents under 15 pages, it takes 1 to 2 minutes. Longer documents can take up to 4 minutes. The AI model runs locally on the server's CPU which is slower than cloud APIs.

Q: What is the difference between Short, Medium, and Long? A: Short gives only the essential key points with minimal text. Medium (the default) gives a balanced overview with section headings and a few sentences per section. Long gives a detailed summary preserving more content, findings, and conclusions.

Q: Can I change the summary length after summarizing?
A: Yes. Click "+ New summary", re-upload your PDF, select a different length, and click Summarize PDF again. Each run is saved separately in your history.

Q: How do I copy my summary?
A: Click the "Copy" button in the summary toolbar. The entire summary — all headings and text — is copied to your clipboard for pasting anywhere.

Q: How do I download my summary?
A: Click "Download PDF" in the summary toolbar. A formatted PDF is saved to your device named "summary-[originalname].pdf".

Q: Are my summaries saved automatically?
A: Yes. Every summary is automatically saved and appears in the History panel on the left sidebar. Click any entry to expand it and see a preview.

Q: How do I delete a past summary?

A: Hover over the item in the History sidebar and click the X button that appears on its right side.

Q: How many past summaries can I see?
A: The history panel shows your 50 most recent summaries, newest first.

Q: Is my original PDF stored on the server?
A: Your PDF is temporarily saved only during processing. Once the summary is generated, the original file is not kept. Only the summary content and basic metadata are stored in the database.

Q: Is my document data private?
A: Yes. The BART model runs on the platform's own local server. Your PDF content is never sent to any third-party service or external API. All processing stays in-house.

Q: What AI model powers the summarization?
A: Facebook AI Research's BART-large-CNN model — a powerful abstractive summarization model that generates new summary text rather than copying sentences from the original.

Q: Does it work for all subjects?
A: Yes — science, law, history, engineering, economics, and more. Quality may vary for highly mathematical documents or non-English PDFs.

Q: What does "Could not extract text from PDF" mean?
A: Your PDF is a scanned document stored as images. You need OCR software to convert it to a text-based PDF before uploading.

Q: Can I summarize Word or PowerPoint files?
A: Not directly. Export them to PDF first using "Save As PDF", then upload the PDF to the summarizer.

Q: Why does my summary have odd headings or seem incomplete? A: Heading detection uses heuristics and may not match every document's structure. Try the Long setting for more detail.
Very long documents may also be partially truncated to avoid server timeouts

Q: Does it work on mobile?
A: Yes, the interface is responsive. Desktop is recommended for the best experience, especially for viewing the history sidebar.

Q: Is there a cost to use this feature?
A: No. It is completely free for all students on the platform. The AI model runs on the platform's own server with no per-use charges.

Q: Can I summarize the same PDF multiple times?
A: Yes. Each run is saved separately. You can use different length settings and compare the results.

Q: What should I do if I get a timeout error?
A: Your document was too long. Try using the Short length setting, or split the file into smaller sections before uploading.

Q: What if I get a general server error?
A: Verify your file is a valid text-based PDF under 50 MB. Refresh the page and try again. If the ML service is not running, the platform administrator needs to restart it.

Q: Does the summarizer need internet to work?
A: No. The BART model runs entirely locally on the server. No internet connection is needed after the initial model download during first setup.

SECTION 9 — PLATFORM INTEGRATION NOTES

ROUTING AND PORTS

- Express backend (Port 5000) should be proxied under a shared API gateway at /api/pdf-summarizer/
- Flask ML service (Port 5001) is internal only — must not be publicly exposed
- React frontend should be embedded at /pdf-summarizer route in the unified platform router

DATABASE

- MongoDB collection: pdfsummaries

- Mongoose model name: PdfSummary
- Database name: pdfSummarizer (capital S — case-sensitive)
- Add a userId field to PdfSummary schema when platform-level authentication is added, so history is user-specific

AUTHENTICATION

Current version has no user authentication. All history is shared across sessions. Must be addressed during integration by linking MongoDB records to the platform's auth system.

PERFORMANCE AND CONCURRENCY

BART is CPU-intensive and handles one inference at a time. For multi-user production, a job queue (Redis Queue or Celery) should be placed in front of the Flask ML service.

7.Notes & To-Do Assistant

Feature Overview

The Notes & To-Do feature allows users to:

Create text-based notes

Create task-based to-do lists

Edit existing notes

Delete notes

Add tags for organization

Mark tasks as completed

There are two types of entries:

Note

Used for writing paragraphs, ideas, or study material.

To-Do

Used for creating task lists where each task can be marked complete.

Creating a Note

Question: How do I create a note?

Chatbot Answer:

To create a note:

Click on **New Note**.

Make sure the **Note** option is selected.

Enter a title.

Write your content in the text area.

Add tags if you want (optional).

Click **Create Note**.

Your note will appear on the main page.

Creating a To-Do List

Question: How do I create a to-do list?

Chatbot Answer:

To create a to-do list:

Click **New Note**.

Select **To-Do**.

Enter a title.

Click **+ Add Item** to add tasks.

Type your task.

Add more items if needed.

Click **Create Note**.

Your to-do list will now be saved.

Editing Notes

Question: How can I edit my note?

Chatbot Answer:

To edit a note:

Click the **Edit** button on the note card.

Update the title, content, tags, or tasks.

Click **Update Note** to save changes.

Deleting Notes

Question: How do I delete a note?

Chatbot Answer:

To delete a note:

Click the **Delete** button on the note.

The note will be permanently removed.

Adding Multiple Tasks

Question: Can I add multiple tasks?

Chatbot Answer:

Yes!
When creating or editing a To-Do list:

Click **+ Add Item** for each new task.

You can add as many tasks as you need.

Marking Tasks as Completed

Question: How do I mark a task complete?

Chatbot Answer:

To mark a task complete:

Click the checkbox next to the task.

The task will be visually marked as completed.

You can uncheck it anytime to mark it incomplete.

Tags System

Question: Can I add tags?

Chatbot Answer:

Yes.
While creating or editing a note:

Enter tags in the Tags field.

Separate multiple tags using commas.

Question: What are tags used for?

Chatbot Answer:

Tags help you organize your notes by topic or category.
They make it easier to identify related notes.

Difference Between Note and To-Do

Question: What is the difference between Note and To-Do?

Chatbot Answer:

A **Note** is for writing paragraphs or detailed information.

A **To-Do** is for creating a checklist of tasks where each item can be marked completed.

Adding Tasks Later

Question: Can I add more tasks later?

Chatbot Answer:

Yes.
Click **Edit** on your To-Do note and:

Click **+ Add Item**

Add new tasks

Click **Update Note**

Removing a Task

Question: Can I remove a task?

Chatbot Answer:

You can remove a task by:

Clicking **Edit** on the To-Do note.

Clearing the task text.

Saving the note again.

Editing Completed Tasks

Question: Can I edit completed tasks?

Chatbot Answer:

Yes.
You can edit any task, whether completed or not:

Click **Edit**

Modify the task text

Save changes

Note Limits

Question: Is there a limit to how many notes I can create?

Chatbot Answer:

You can create multiple notes and to-do lists.
There is no visible limit within normal usage.

Leaving Tags Empty

Question: What happens if I leave tags empty?

Chatbot Answer:

Nothing happens.
Tags are optional.
Your note will still be saved without tags.

Switching Note Type

Question: Can I switch a note to a to-do?

Chatbot Answer:

You can:

Click **Edit**

Switch between **Note** and **To-Do**

Adjust content accordingly

Save changes

No Notes Showing

Question: I don’t see any notes. What should I do?

Chatbot Answer:

If no notes are showing:

Click **Create your first Note**

Add a new note or to-do

Save it

Your first note will appear on the main page.

Organizing Notes

Question: How do I organize my notes?

Chatbot Answer:

You can organize notes by:

Using meaningful titles

Adding tags

Separating content into different notes

Creating To-Do lists for task management

Tracking Tasks

Question: How can I keep track of tasks?

Chatbot Answer:

Use the To-Do feature:

Add tasks

Mark them complete when finished

Review completed vs pending items visually

SECTION — APPLICATION WORKFLOW (DEMO VIDEO EXPLANATION)

1. AI Learning Assistant – Complete Workflow

Step 1 – Create an Account

Open the platform in the browser.

Click **Sign Up**.

Enter:

- Name
- Email
- Password (minimum 6 characters)

Click **Create Account**.

The user is automatically logged in.

Step 2 – Upload a Document

Go to the **Dashboard**.

Click **Upload Document**.

Select a **PDF file (max 10MB)**.

Click **Upload**.

System actions:

Text is extracted from the PDF.

The document is stored in the user library.

Step 3 – Open the Document

The uploaded document appears in the **document library**.

Click the document name.

The document viewer opens.

Step 4 – Chat with AI

Click **Chat with AI**.

Type a question in the chat box.

Click **Send**.

Example questions:

"Explain the main concept."

"What is the conclusion of the paper?"

System workflow:

Relevant document sections are retrieved.

AI generates a contextual answer.

The response appears instantly.

Conversation history is automatically saved.

Step 5 – Generate Summary

Open the document.

Click **Generate Summary**.

System process:

AI reads the document.

Key points are extracted.

A concise summary is generated.

The summary appears on screen and is saved for future access.

Step 6 – Generate Flashcards

Open a document.

Click **Generate Flashcards**.

Choose the number of flashcards (5–50).

Click **Generate**.

System process:

AI analyzes the document.

Important concepts are converted into **question–answer flashcards**.

User interaction:

Click a card to flip it.

Mark favorites.

Navigate with **Next / Previous**.

Step 7 – Generate Quiz

Open the document.

Click **Generate Quiz**.

Choose number of questions (5–20).

Click **Generate Quiz**.

System process:

AI creates multiple-choice questions.

Output includes:

4 answer options

Correct answer

Explanation

Users receive an **instant score after submission**.

2. PDF Summarizer – Complete Workflow

Step 1 – Open the PDF Summarizer

Navigate to the **PDF Summarizer section**.

Step 2 – Upload a PDF

Users can upload in two ways:

Method A – Drag & Drop

Drag the PDF file.

Drop it into the upload area.

Method B – Browse

Click **Browse Files**.

Select a PDF file.

Click **Open**.

Only **PDF files under 50MB** are accepted.

Step 3 – Select Summary Length

Choose the level of detail:

Short

Medium

Long

Step 4 – Start Summarization

Click **Summarize PDF**.

System workflow:

Frontend sends file to backend.

Backend sends it to the ML service.

Text is extracted using PyPDF2.

The BART model processes the text.

Summaries are generated.

Step 5 – View the Summary

The output appears as:

Structured sections

Headings

Summary paragraphs

Step 6 – Download the Summary

Click **Download PDF**.

A summarized PDF file is saved to the device.

3. Video Summarization – Complete Workflow

Step 1 – Open Video Section

Navigate to:

```
Application → Video Section
```

Step 2 – Upload Video

Two upload options are available.

Drag & Drop

Drag video file (.mp4, .mov, .avi).

Drop into upload area.

Browse Files

Click **Browse Files**.

Select video.

Click **Open**.

Step 3 – System Processing

After upload:

Audio is extracted from the video.

Speech-to-text conversion creates a transcript.

AI analyzes the transcript.

A textual summary is generated.

Step 4 – View Summary

The generated summary appears below the upload section.

4. YouTube Video Question Answering – Workflow

Step 1 – Enable YouTube Mode

Navigate to **Video Section**.

Turn ON **Enable YouTube Video Question Answering**.

Step 2 – Enter Video URL

Copy the YouTube video link.

Paste it into the URL input field

Step 3 – Load Video Data

Click **Load / Process**.

The system retrieves the video transcript.

Step 4 – Ask Questions

Type a question.

Click **Ask**.

Example:

```
What is the main topic of this video?
```

Step 5 – View Answer

The AI processes the transcript and generates an answer.

5. Resume Builder – Complete Workflow

1. User Authentication

Before accessing the Resume Builder, the user must be authenticated.

Step 1 – Open the Platform

The user opens the platform in a browser.

The landing page displays login and registration options.

Step 2 – Register or Login

Registration Process

Click **Register**.

Enter:

Name

Email

Password

Click **Create Account**.

System actions:

Password is hashed using **bcrypt**.

User data is stored in the database.

A **JWT token** is generated.

Login Process

Enter email and password.

Click **Login**.

2. Resume Dashboard

After login, the user is redirected to the **Resume Dashboard**, which acts as the central hub for resume management.

Dashboard Features

The dashboard displays:

Resume cards

Resume title

Last updated date

Quick action icons

Users can perform the following actions:

Create a new resume

Upload an existing resume

Edit an existing resume

Rename a resume

Delete a resume

3. Creating a New Resume

Step 1 – Start Resume Creation

Click **Create Resume**.

A dialog box appears asking for the resume title.

Enter a title.

Example:

```
Software Developer Resume
```

Click **Create**.

System actions:

A new resume document is created in the database.

Default fields are initialized.

The user is redirected to the **Resume Builder editor**.

4. Resume Builder Editor Interface

The Resume Builder editor is a **split-screen interface**.

Left Panel – Form Editor

Contains structured input forms where the user enters resume data.

Right Panel – Live Resume Preview

Displays a real-time preview of the resume layout.

Every time the user updates a field, the preview updates instantly.

5. Resume Form Sections

The resume form is divided into **six main sections**.

Navigation buttons:

Previous

Next

These allow the user to move between sections.

Section 1 – Personal Information

This section collects the user's basic details.

Fields include:

- Full Name
- Job Title / Profession
- Email Address
- Phone Number
- Location
- Website URL
- LinkedIn Profile
- Profile Photo

Profile Photo Upload

Steps:

- Click **Upload Photo**.
- Select an image file.
- Click **Save Changes**.

System processing:

- Image is uploaded to **ImageKit CDN**.
- The image is face-cropped automatically.
- Optional AI background removal can be applied.
- The processed image URL is saved in the database.

Section 2 – Professional Summary

This section contains a short paragraph summarizing the user's career.

Typical length:

3–5 sentences.

AI Summary Enhancement

Users can click **Enhance with AI**.

System process:

The text is sent to the AI endpoint.

The AI rewrites the paragraph.

A polished professional summary is returned.

The user can edit the generated text if needed.

Section 3 – Work Experience

Users can add multiple work experience entries.

Each entry includes:

Company Name

Job Title

Start Date

End Date

Job Description

Adding Experience

Click **Add Experience**.

Fill the fields.

Click **Save**.

Currently Working Option

If the user is still employed:

Check **Currently Working Here**.

End date becomes disabled.

AI Job Description Enhancement

Users can improve their job descriptions using AI.

Steps:

Click **Enhance with AI**.

The AI analyzes:

Company

Position

Description

The system generates an improved description using stronger action verbs.

Section 4 – Education

Users can add multiple education entries.

Fields include:

Institution Name

Degree

Field of Study

Graduation Date

GPA (optional)

Section 5 – Projects

Users can add project entries to showcase their work.

Each project contains:

- Project Name
- Project Type
- Description

Multiple projects can be added.

Section 6 – Skills

Users add skills as **tag elements**.

Example skills:

- Python
- Machine Learning
- React
- SQL
- TensorFlow

Each skill appears as a **colored pill badge** in the preview.

6. Resume Templates

Users can choose between different resume templates.

Available templates:

- Modern
- Classic
- Minimal

Minimal with Image

Template Switching

Steps:

Click **Template Selector**.

Choose a template.

System actions:

Resume preview re-renders instantly.

No data is lost.

7. Accent Color Customization

Users can customize the visual style of the resume.

Steps:

Open the **Color Picker**.

Select a color.

The selected color is applied to:

Header

Section titles

Skill badges

Borders

8. Saving Resume Data

To save progress:

Click **Save Changes**.

9. Resume Visibility Settings

Resumes are private by default.

Making Resume Public

Click the **Private/Public toggle**.

Set visibility to **Public**.

System actions:

Resume becomes accessible via a shareable link.

Example link:

`/view/resumeID`

10. Sharing Resume

Steps:

Set resume visibility to **Public**.

Click **Share**.

System actions:

The Web Share API generates a public URL.

Users can send it via email, WhatsApp, or LinkedIn.

11. Downloading Resume as PDF

Steps:

Click **Download**.

System process:

Browser print function is triggered.

CSS print rules format the resume.

The user selects **Save as PDF**.

The resume is downloaded to the device.

12. Editing Existing Resumes

Users can modify resumes anytime.

Steps:

Open the **Dashboard**.

Click a resume card.

Edit sections.

Click **Save Changes**.

13. Deleting a Resume

Steps:

Hover over the resume card.

Click the **Trash icon**.

Confirm deletion.

The resume is permanently removed.

6. Notes & To-Do Assistant – Complete Workflow

Creating a Note

Click **New Note**.

Select **Note**.

Enter a title.

Write content.

Add tags (optional).

Click **Create Note**.

The note appears on the main page.

Creating a To-Do List

Click **New Note**.

Select **To-Do**.

Enter title.

Click **+ Add Item**.

Add tasks.

Click **Create Note**.

Editing a Note

Click **Edit**.

Modify content.

Click **Update Note**.

Deleting a Note

Click **Delete**.

The note is permanently removed.

Marking Tasks Complete

Click the **checkbox** next to the task.

The task is marked completed.

Detailed Troubleshooting Guide for All Features

Resume Builder – Troubleshooting Guide

This allows your chatbot to answer **user problems directly**.

Problem 1 – Profile Photo Not Uploading

Possible Causes

Unsupported image format

File size too large

Network issue during upload

ImageKit server not responding

User clicked save before selecting image

Solution

Ensure the image format is supported:

JPG

PNG

JPEG

Ensure the file size is small (recommended under **5MB**).

Check internet connection.

Try uploading the image again.

Click **Save Changes** after selecting the photo.

Technical Explanation

When the user uploads a profile photo:

- The image is temporarily stored in frontend state.
- When **Save Changes** is clicked, the image is sent to the backend.
- The backend uploads the image to **ImageKit CDN**.
- The returned URL is stored in the database.

If the upload fails, the image will not appear in the resume preview.

Problem 2 – AI Enhance Button Not Working

Cause

Required fields are empty.

Solution

Before clicking **Enhance with AI**, make sure:

- Company Name is filled
- Job Title is filled

Technical Reason

The frontend blocks the API request if required fields are missing.

Problem 3 – Resume Not Downloading

Cause

The browser blocked the print dialog.

Solution

- Allow pop-ups in the browser.

Click **Download** again.

Choose **Save as PDF** in the print dialog.

Problem 4 – Public Resume Link Not Working

Cause

Resume visibility is still set to **Private**.

Solution

Open Resume Builder.

Toggle **Private → Public**.

Click **Share** again.

Problem 5 – Resume Data Not Saving

Cause

User forgot to click **Save Changes**.

Solution

Always click **Save Changes** before leaving the page.

Why This Is Important for Your Chatbot

Without these **problem-solution sections**, the chatbot will answer:

> "I don't know."

Because **RAG models only answer from the knowledge base**.

This section lists common user issues, possible causes, solutions, and technical explanations for each feature of the application. It is designed so the chatbot can answer both simple user questions and technical support questions.

---

1. Global / Platform-Level Issues

Problem: Backend connection failed

Possible Causes

1.

FastAPI server is not running.

2.
3.

Incorrect API endpoint is configured in the frontend.

4.
5.

CORS is not enabled.

6.
7.

Backend crashed due to an internal error.

8.

Solution

1.

Start the backend server:
```
uvicorn main:app --reload
```

2.
3.

Verify that the frontend API URL is correct.

4.

5.

Ensure CORS middleware is enabled.

6.
7.

Check backend terminal logs for runtime errors.

8.

Technical Explanation

The frontend sends requests to the backend API. If the backend is not running, the URL is wrong, or CORS blocks the request, the application cannot communicate with the server.

Problem: Chatbot says “I don't know”

Possible Causes

1.

The answer is not present in the uploaded documentation.

2.
3.

The vector store was not rebuilt after updating documentation.

4.
5.

The wrong documentation PDF was loaded.

6.
7.

Relevant chunks were not embedded properly.

8.

Solution

1.

Ensure the latest consolidated documentation PDF is uploaded.

2.
3.

Rebuild the vector store / FAISS index.

4.
5.

Restart the backend server.

6.
7.

Re-test with a question that is clearly covered in the documentation.

8.

Technical Explanation

The RAG pipeline retrieves top matching chunks from the indexed documentation. If the requested information is missing from the embedded content, the chatbot cannot answer.

---

2. AI Learning Assistant

Problem: Cannot create account

Possible Causes

1.

Password is too short.

2.

3.

Email format is invalid.

4.
5.

Email is already registered.

6.
7.

Backend auth service is unavailable.

8.

Solution

1.

Use a password with at least 6 characters.

2.
3.

Enter a valid email address.

4.
5.

Try logging in if the account already exists.

6.
7.

Refresh the page and try again.

8.

Technical Explanation

The registration flow validates user input before account creation. Invalid fields or duplicate records prevent user creation.

Problem: Login not working

Possible Causes

1.

Incorrect email or password.

2.
3.

Expired login session.

4.
5.

Invalid token stored in browser.

6.
7.

Auth API is failing.

8.

Solution

1.

Re-enter correct credentials.

2.
3.

Clear browser storage and log in again.

4.
5.

Refresh the page.

6.
7.

Check whether the backend auth service is running.

8.

Technical Explanation

The application uses authentication tokens to keep users logged in. If the token is invalid or expired, protected routes fail and login must be repeated.

Problem: PDF upload failed

Possible Causes

1.

File is larger than 10MB.

2.
3.

File is not a PDF.

4.
5.

PDF is scanned/image-based.

6.
7.

Upload was interrupted.

8.

Solution

1.

Upload a text-based PDF under 10MB.

2.
3.

Make sure the file extension is `.pdf`.

4.
5.

Convert scanned PDFs using OCR before uploading.

6.
7.

Retry the upload.

8.

Technical Explanation

The feature extracts readable text from PDFs. If the file is image-based or too large, extraction fails and AI features cannot work.

Problem: Document opens but AI cannot answer questions

Possible Causes

1.

Text extraction failed or was incomplete.

2.
3.

The requested topic is not clearly present in the document.

4.
5.

Retrieval did not find the relevant chunk.

6.
7.

The document upload was corrupted.

8.

Solution

1.

Re-upload the document.

2.
3.

Test with a direct question from the document.

4.
5.

Use a cleaner text-based PDF.

6.
7.

Regenerate the document index if needed.

8.

Technical Explanation

Document chat depends on extracted text and retrieval. If either extraction or retrieval fails, answer quality drops.

Problem: AI response is slow

Possible Causes

1.

Large document size.

2.
3.

API latency.

4.
5.

Heavy processing on the server.

6.
7.

Slow internet connection.

8.

Solution

1.

Wait 10–20 seconds.

2.
3.

Try a smaller document.

4.
5.

Check your internet connection.

6.
7.

Retry later if the server is under load.

8.

Technical Explanation

Larger documents increase retrieval and processing time, especially when multiple AI features run on the same server.

Problem: Summary not generating

Possible Causes

1.

PDF text could not be extracted.

2.
3.

AI service is unavailable.

4.
5.

Document is too large or malformed.

6.
7.

Backend request failed.

8.

Solution

1.

Re-upload a text-based PDF.

2.
3.

Try again with a smaller file.

4.
5.

Refresh the page and retry.

6.
7.

Check backend and AI service logs.

8.

Technical Explanation

Summary generation depends on successful text extraction plus a working AI inference path. Failure in either stage stops summary creation.

Problem: Flashcards not generating

Possible Causes

1.

Document has insufficient extractable text.

2.
3.

AI generation failed.

4.
5.

Requested flashcard count is too high for available content.

6.
7.

Temporary server issue.

8.

Solution

1.

Re-upload the document.

2.
3.

Try generating fewer flashcards first.

4.
5.

Use a document with clearer textual content.

6.
7.

Retry after refreshing the page.

8.

Technical Explanation

Flashcards are generated from document concepts. Poor extraction or insufficient content reduces output quality.

Problem: Quiz not generating

Possible Causes

1.

Document content is too short.

2.
3.

AI service failed to create MCQs.

4.
5.

Retrieval from the document was weak.

6.
7.

Backend request timed out.

8.

Solution

1.

Try a longer document.

2.
3.

Reduce the question count.

4.
5.

Refresh and retry.

6.
7.

Check the backend service.

8.

Technical Explanation

Quiz generation requires enough meaningful content to produce questions, answer options, and explanations.

---

3. Resume Builder

Problem: Cannot login

Possible Causes

1.

Incorrect password.

2.
3.

Expired token.

4.
5.

Invalid token in localStorage.

6.
7.

Auth middleware rejected the request.

8.

Solution

1.

Re-enter correct credentials.

2.
3.

Clear localStorage.

4.
5.

Login again.

6.

Technical Explanation

The Resume Builder uses JWT authentication. Invalid or expired tokens cause protected API requests to return 401 Unauthorized.

Problem: Dashboard not showing resumes

Possible Causes

1.

Token validation failed.

2.
3.

Resume fetch request failed.

4.
5.

No resumes exist yet.

6.
7.

Backend is unavailable.

8.

Solution

1.

Refresh the page.

2.
3.

Log in again.

4.
5.

Create a new resume if the dashboard is empty.

6.
7.

Check backend connectivity.

8.

Technical Explanation

The dashboard loads resumes from the backend after user validation. If user state is not restored correctly, cards may not appear.

Problem: Cannot create a new resume

Possible Causes

1.

Title field was left empty.

2.
3.

Create request failed.

4.
5.

User session expired.

6.
7.

Database save error occurred.

8.

Solution

1.

Enter a valid resume title.

2.
3.

Retry after refreshing the page.

4.
5.

Log in again if session expired.

6.
7.

Check backend logs.

8.

Technical Explanation

Creating a new resume initializes a document in MongoDB. Invalid auth or request failures stop this flow.

Problem: Existing resume upload is not working

Possible Causes

1.

Uploaded file is not a valid PDF.

2.
3.

PDF parsing failed.

4.
5.

AI extraction service failed.

6.
7.

Uploaded text could not be structured into resume fields.

8.

Solution

1.

Upload a readable PDF resume.

2.
3.

Retry with a cleaner, text-based resume PDF.

4.
5.

Wait for the loading spinner to finish.

6.
7.

Try manual resume creation if parsing fails.

8.

Technical Explanation

The system extracts raw PDF text and sends it to AI for schema-based field extraction. If parsing or AI structuring fails, auto-fill cannot complete.

Problem: AI enhance button is disabled

Possible Causes

1.

Company name is empty.

2.
3.

Job title is empty.

4.
5.

Frontend validation is blocking the request.

6.

Solution

1.

Fill in Company Name.

2.
3.

Fill in Job Title.

4.
5.

Try the AI button again.

6.

Technical Explanation

The frontend intentionally disables AI enhancement until required context fields are present.

Problem: Professional summary AI is not working

Possible Causes

1.

Auth token is missing.

2.
3.

AI endpoint is unavailable.

4.
5.

Request failed due to network issue.

6.
7.

Input is empty and backend rejected the request.

8.

Solution

1.

Refresh and log in again.

2.
3.

Enter a rough draft first.

4.
5.

Retry the AI action.

6.
7.

Check backend AI route availability.

8.

Technical Explanation

Summary enhancement uses a protected AI endpoint. It requires an authenticated request and a responsive backend.

Problem: Profile photo is not uploading

Possible Causes

1.

User selected an unsupported image file.

2.
3.

Save Changes was not clicked after selecting the image.

4.
5.

Upload to ImageKit failed.

6.
7.

Network interruption occurred.

8.
9.

Backend image handling failed.

10.

Solution

1.

Use a standard image format such as JPG or PNG.

2.
3.

Select the image again.

4.
5.

Click **Save Changes** after choosing the image.

6.
7.

Retry on a stable internet connection.

8.
9.

Check backend logs if the issue continues.

10.

Technical Explanation

The selected image is first held in frontend state. It is uploaded only when the user clicks **Save Changes**. The backend then sends it to ImageKit, receives a CDN URL, and stores that URL in the database. If any step fails, the image will not appear.

Problem: Resume changes are not saving

Possible Causes

1.

Save button was not clicked.

2.
3.

JWT token expired.

4.
5.

PUT request failed.

6.
7.

Database write error occurred.

8.

Solution

1.

Click **Save Changes** again.

2.
3.

Log in again.

4.
5.

Refresh the page and retry.

6.
7.

Check backend logs for update errors.

8.

Technical Explanation

All resume data, including sections, template, accent color, visibility, and image URL, is persisted only through the update API request.

Problem: Template is not changing

Possible Causes

1.

State update failed.

2.
3.

Preview rendering error occurred.

4.
5.

Save was not performed after changing template.

6.

Solution

1.

Select the template again.

2.
3.

Check whether the live preview changes.

4.
5.

Click **Save Changes**.

6.

Technical Explanation

The preview renders a different React template component based on the selected template state. If state is not stored, the old template remains.

Problem: Accent color is not updating

Possible Causes

1.

Color picker selection was not saved.

2.
3.

Preview did not re-render.

4.
5.

Save request failed.

6.

Solution

1.

Re-select the color.

2.
3.

Verify the preview updates instantly.

4.
5.

Click **Save Changes**.

6.

Technical Explanation

Accent color is stored in `resumeData.accent_color` and passed to the active template as a prop. If save fails, the old value persists.

Problem: Resume not downloading

Possible Causes

1.

Browser blocked print dialog.

2.
3.

Pop-ups are disabled.

4.
5.

User did not choose “Save as PDF”.

6.
7.

Browser print feature malfunctioned.

8.

Solution

1.

Allow pop-ups / print dialog.

2.
3.

Click **Download** again.

4.
5.

In the print window, choose **Save as PDF**.

6.

Technical Explanation

The download feature uses `window.print()` with print CSS, not a separate file-generation endpoint. If the browser blocks printing, no PDF is produced.

Problem: Public link not working

Possible Causes

1.

Resume visibility is still Private.

2.
3.

Share link was generated before toggling public.

4.
5.

Public fetch API did not return the resume.

6.

Solution

1.

Open the resume in the builder.

2.
3.

Toggle visibility from **Private** to **Public**.

4.
5.

Click **Share** again.

6.

Technical Explanation

The public route only returns resumes where `public: true`. Private resumes are intentionally inaccessible from public URLs.

Problem: Share button not appearing

Possible Causes

1.

Resume is still Private.

2.
3.

Builder state did not refresh after toggle.

4.
5.

Save/update request failed.

6.

Solution

1.

Set the resume to **Public**.

2.
3.

Save changes if needed.

4.
5.

Refresh the builder page.

6.

Technical Explanation

The Share button appears only when the resume visibility is public.

---

4. Video Summarization

Problem: Video file upload is not working

Possible Causes

1.

Unsupported file type.

2.
3.

File selection failed.

4.
5.

Upload interrupted.

6.
7.

Frontend or backend upload error.

8.

Solution

1.

Use common supported formats such as `.mp4`, `.mov`, or `.avi`.

2.
3.

Retry using Browse Files.

4.
5.

Ensure the file is not corrupted.

6.
7.

Refresh and upload again.

8.

Technical Explanation

The feature expects a valid local video file so the system can extract audio and create a transcript.

Problem: Video uploaded but no summary appears

Possible Causes

1.

Audio extraction failed.

2.
3.

Speech-to-text transcription failed.

4.
5.

Summarization model failed.

6.
7.

Processing is still ongoing.

8.

Solution

1.

Wait for processing to complete.

2.
3.

Re-upload the video.

4.
5.

Try a clearer video with audible speech.

6.
7.

Check backend logs.

8.

Technical Explanation

The file workflow is sequential: upload → audio extraction → transcription → summarization → display. Failure at any stage prevents summary output.

Problem: Wrong mode selected in video section

Possible Causes

1.

YouTube QA toggle was ON during file summarization.

2.
3.

User expected file upload and YouTube mode to work together.

4.

Solution

1.

Keep **Enable YouTube Video Question Answering** OFF when summarizing local video files.

2.
3.

Use file upload only for local video summarization.

4.

Technical Explanation

Local file summarization and YouTube QA are separate workflows and should not be mixed.

Problem: Transcript quality is poor

Possible Causes

1.

Video audio is unclear.

2.

3.

Background noise is high.

4.
5.

Speaker volume is too low.

6.
7.

Multiple overlapping speakers exist.

8.

Solution

1.

Use a clearer video.

2.
3.

Prefer videos with clean speech.

4.
5.

Retry with a shorter, higher-quality clip.

6.

Technical Explanation

Summary quality depends heavily on the speech-to-text transcript quality.

---

5. YouTube Video Question Answering

Problem: URL field is not appearing

Possible Causes

1.

YouTube mode toggle is OFF.

2.

Solution

1.

Turn ON **Enable YouTube Video Question Answering**.

2.

Technical Explanation

The URL input field is shown only after enabling YouTube QA mode.

Problem: YouTube video is not loading

Possible Causes

1.

Invalid YouTube URL.

2.
3.

Transcript could not be retrieved.

4.
5.

Video has no accessible transcript.

6.
7.

Backend processing failed.

8.

Solution

1.

Paste a valid YouTube URL.

2.
3.

Try another video.

4.
5.

Reload and click **Load / Process** again.

6.

Technical Explanation

The feature depends on fetching a usable transcript from the YouTube video before answering questions.

Problem: Question answering is not working

Possible Causes

1.

Video was not processed first.

2.
3.

Transcript fetch failed.

4.
5.

Question input is empty.

6.
7.

QA model request failed.

8.

Solution

1.

Load/process the video before asking.

2.
3.

Type a clear question.

4.
5.

Retry after the video transcript is ready.

6.

Technical Explanation

The QA model answers from the loaded transcript. If the transcript is missing, there is no knowledge source for answering.

Problem: “Summarize the video” is not working

Possible Causes

1.

Video transcript not loaded.

2.
3.

User asked before clicking Load / Process.

4.
5.

QA flow failed after transcript retrieval.

6.

Solution

Enable YouTube mode.

Paste the link.

Click **Load / Process**.

Ask: “Summarize the video.”

Technical Explanation

YouTube summarization uses the QA workflow over the fetched transcript, not the local file summarization flow.

6. PDF Summarizer

Problem: PDF not uploading

Possible Causes

File is larger than 50MB.

File is not a PDF.

Multer upload filter rejected the file.

Upload was interrupted.

Solution

Upload a PDF under 50MB.

Verify the file extension is `.pdf`.

Retry the upload.

Technical Explanation

The upload middleware accepts only PDFs and enforces a 50MB limit.

Problem: “Could not extract text from PDF”

Possible Causes

PDF is scanned or image-based.

Text layer is missing.

PDF is malformed.

Solution

Try selecting text in a PDF viewer.

If text cannot be selected, use OCR to convert the file.

Upload the OCR-converted PDF.

Technical Explanation

The summarizer depends on extractable text. Image-only PDFs cannot be summarized directly.

Problem: Summarization is taking too long

Possible Causes

Document is large.

BART is running on CPU.

The server is under load.

Long summary setting increases processing time.

Solution

Wait longer for completion.

Use **Short** summary length.

Split very large PDFs into smaller files.

Retry during lower server load.

Technical Explanation

BART-large-CNN runs locally on CPU, so processing time increases with page count and chunk volume.

Problem: Timeout error

Possible Causes

PDF is too long, usually 30+ pages.

Server processing exceeded time limit.

Long setting generated too much inference work.

Solution

Use **Short** summary length.

Split the document into smaller sections.

Retry with fewer pages.

Technical Explanation

Very long documents may exceed the server timeout during chunk-based summarization.

Problem: Summary quality is poor or incomplete

Possible Causes

Heading detection was inaccurate.

Document formatting is unusual.

Mathematical content extracted badly.

Text was truncated to avoid timeout.

Solution

Try **Long** summary length.

Use a cleaner text-based PDF.

Split heavily formatted documents into smaller parts.

Technical Explanation

Headings are detected heuristically, and mathematical or unusual layouts may reduce output quality.

Problem: Download PDF is not working

Possible Causes

Summary generation failed.

Download response was interrupted.

Backend did not return the generated PDF properly.

Solution

Confirm the summary appears first.

Click **Download PDF** again.

Retry after refreshing the page.

Technical Explanation

The system generates a formatted summary PDF using ReportLab and returns it after successful summarization.

Problem: Copy to clipboard is not working

Possible Causes

Browser clipboard permission issue.

Summary content was not rendered yet.

UI interaction failed.

Solution

Wait until the summary is fully visible.

Click **Copy** again.

Ensure browser permissions are enabled.

Technical Explanation

Clipboard actions depend on browser support and rendered summary content.

Problem: History panel not showing old summaries

Possible Causes

History fetch failed.

Record count exceeded visible range.

Database connection issue

Solution

Refresh the page.

Check if summaries were generated successfully.

Confirm backend database connection.

Technical Explanation

The history panel shows the 50 most recent summary records from MongoDB.

Problem: Delete history item is not working

Possible Causes

Delete request failed.

UI state did not refresh.

Database deletion failed.

Solution

Click delete again.

Refresh the history panel.

Check backend logs for delete errors.

Technical Explanation

History deletion calls the backend delete route and then updates the UI list.

7. Notes & To-Do Assistant

Problem: Cannot create a note

Possible Causes

Title was not entered.

Content was left empty when required.

Create request failed.

Backend/database issue occurred.

Solution

Enter a title.

Add content or tasks based on note type.

Click **Create Note** again.

Refresh and retry.

Technical Explanation

A note or to-do is created only after the form data is submitted successfully to the backend.

Problem: To-do list is not saving

Possible Causes

No title entered.

Tasks were not added correctly.

Create request failed.

Solution

Enter a title

Click + **Add Item** and type tasks.

Click **Create Note** again.

Technical Explanation

A to-do list is stored as a titled record containing an array of task items.

Problem: Edit button not working

Possible Causes

UI state failed to open edit mode.

Existing note data did not load.

Update request failed after editing.

Solution

Click **Edit** again.

Refresh the page.

Retry the edit.

Technical Explanation

Editing requires loading the existing note into editable form state before submitting an update.

Problem: Note changes not updating

Possible Causes

Update Note button was not clicked.

Update request failed.

Backend save error occurred.

Solution

Make edits again.

Click **Update Note**.

Refresh and verify.

Technical Explanation

Edits are not persisted until the update action is submitted.

Problem: Delete note not working

Possible Cause

Delete request failed.

UI did not refresh after deletion.

Backend/database issue.

Solution

Click **Delete** again.

Refresh the page.

Check backend connectivity.

Technical Explanation

Delete removes the note permanently from storage.

Problem: Cannot add more tasks

Possible Causes

User did not click **+ Add Item**.

Form state issue occurred.

Edit mode was not opened for an existing to-do

Solution

Click **+ Add Item** for each new task.

Open the note in edit mode if it already exists.

Save the updated note afterward.

Technical Explanation

Each task is added as another item in the to-do items array.

Problem: Task checkbox is not marking completion

Possible Causes

Click event failed.

UI state update did not persist.

Backend update failed.

Solution

Click the checkbox again.

Refresh the page.

Re-open the to-do list and verify

Technical Explanation

Task completion depends on toggling the item's completed state in the stored list.

Problem: Tags are not saving

Possible Causes

Tags were not entered correctly.

Save/update action was not completed.

Backend rejected the update.

Solution

Enter tags separated by commas.

Save the note again.

Refresh and verify.

Technical Explanation

Tags are stored as comma-separated values that are processed into an organizational list.

Problem: No notes are showing

Possible Causes

No notes have been created yet.

Notes fetch failed.

Backend/database connection issue.

Solution

Click **Create your first Note**.

Add a new note or to-do.

Refresh the page.

Technical Explanation

An empty screen may simply indicate no stored notes yet, or it may mean the fetch request failed.